

Checkpoint IV: Second Prototype

Group: G16

Date: 2025/10/05

Prototype Architecture

The goal of the project is to enable **comparative analysis of Pokémon card prices** across multiple **sets, years, series, and types**.

The visualization supports collectors, traders, and researchers who want to explore how **market dynamics** and **card characteristics** affect value and popularity over time.

Each idiom in the dashboard highlights a different perspective of this analytical theme:

- The **scatterplot** shows the relationship between card price and sales volume, revealing popularity-vs-value patterns.
- The **bar chart** ranks the **top-selling cards**, giving a clear snapshot of market leaders.
- The **line chart** displays **average price variation by condition** (mint → poor), illustrating how quality affects value for each card.
- The planned **violin plot** (for Checkpoint V) will show **price distribution per card type**, emphasizing statistical spread and outliers.

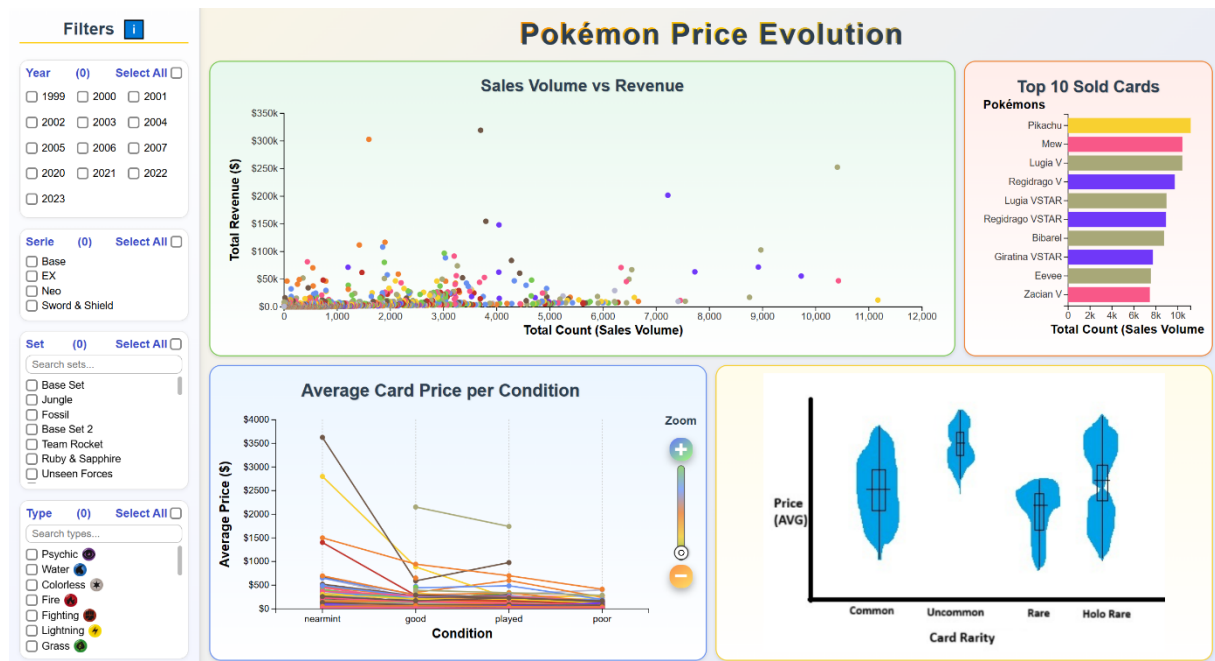
Together, these idioms form a coherent, multi-view system for comparing Pokémon cards across several categorical and temporal dimensions.

The architecture was extended from a single-idiom prototype into a multi-idiom, preprocessed dashboard, with emphasis on efficiency and coordinated interaction. Key changes are:

1. **Global State**
 - Selection now uses a `selectedCards` array (instead of a single `selectedCard` object), enabling multi-card comparisons.
 - `updateSelectionAcrossPlots()` was generalized to handle scatterplot circles, bar chart bars, and line chart paths, ensuring consistent highlighting across idioms.
2. **Preprocessing Pipeline**
 - Data for scatterplot, bar chart and line chart is now pre-aggregated at load time (`scatterTable` (reused for the bar chart) and `lineTable`).
 - This avoids recomputing metrics (totals, averages, condition normalization) after every filter change, significantly improving responsiveness.
 - Filters now only slice the pre-processed tables rather than recalculate from raw CSV.
3. **Expanded Idiom Modules**
 - **Bar chart** and **line chart** are fully implemented and integrated alongside the scatterplot.
 - Each chart has its own module (`createBarchart`, `createLineChart`, etc.), keeping the system modular and extensible.
 - `updateCharts()` orchestrates updates across all idioms using the same filtered subset of data.
4. **Integration Mechanism**
 - Linked highlighting is now bidirectional: selections in scatterplot, bar chart, or line chart propagate across all views.
 - Global deselection (click outside elements) resets the state consistently in all charts.
5. **Usability Enhancements**
 - Added filter count indicators, “select all” logic with lock to prevent rapid refreshes, and a search bar inside filter groups.

- New tooltip/info icon provides interaction hints, improving user onboarding.

Dashboard Layout



The overall layout (left-hand filters and right-hand visualization grid) remains unchanged from CP III, but two refinements were added:

1. **Year Filters** – years are now displayed in parallel columns, reducing scrolling and empty space.
2. **Autocomplete Search** – filters with many options (e.g., sets, types) now include search bars with autocomplete, helping users quickly locate specific filters.
3. **Information Icon** – a help icon with tooltip was added to provide users with interaction hints.

Aside from these improvements, the chart areas remain the same, with scatterplot and bar chart on the top row, line chart and violin plot placeholder on the bottom row.

Data Processing

In this checkpoint, we expanded preprocessing to support two new idioms:

- Line Chart which answers the Task 3: How does condition affect price in relation to set, type and year?
- Bar Chart which answers the Task 1: Are there any favourite or more popular cards per set and per type?

The biggest improvement since CP III is the introduction of **preprocessing pipelines** at data load, which avoids recalculating metrics after each filter change and makes interactions more responsive.

1. Scatterplot & Bar Chart

- Data is pre-aggregated with d3.rollup into a scatterTable.
- For each card (name–series–set), total sales volume, total revenue, type, and release year are stored.

- The **bar chart** reuses this table and applies an additional aggregation and ranking step to extract the **Top 10 most sold cards** for visualization.

2. Line Chart

- A second table (lineTable) is precomputed, normalizing card **conditions** (mint → poor) and storing average prices per condition.
- Values are sorted in condition order, producing consistent multi-line arrays ready for plotting.

3. Efficiency Gains

- Unlike CP III, where metrics were recomputed after each filter change, CP IV builds **scatterTable** and **lineTable** once during data load. At runtime filtering now only slices these pre-aggregated tables, ensuring faster updates and smoother interactivity.
- This ensures fast updates and smooth transitions even when multiple idioms are active.

Chart Interaction

Interactivity was significantly deepened in Checkpoint IV.

Both the bar chart and line chart were restructured to follow D3's **Enter–Update–Exit** pattern—an essential idiom for scalable, dynamic visualization.

Bar Chart: “Top 10 Sold Cards”

Purpose:

- Display the top-selling Pokémon cards across the currently selected filters.
It emphasizes comparative magnitude through horizontal bar lengths.

Enter Phase:

- When the filtered dataset changes, new bar elements are created only for cards that were not previously displayed.
Each new bar starts with zero width and low opacity, gradually expanding to its full size during the animation.
This design choice allows users to perceive new entries as appearing *into* the ranking, rather than abruptly replacing others.

Update Phase:

- Existing bars smoothly adjust their lengths when the underlying count values change (for example, when filters modify total sales).
The chart title and axes also transition to reflect the new maximum scale dynamically.
Because updates occur with transitions rather than re-rendering, the viewer can visually track how each bar grows, shrinks, or reorders.

Exit Phase:

- Bars representing cards no longer in the Top 10 fade out and are removed.
This avoids clutter and maintains focus on relevant data.
A short fade duration provides a perceptual cue that elements are leaving intentionally, rather than disappearing due to error.

Line Chart: “Average Card Price per Condition”

Purpose:

- To show how average card price changes with card condition for each Pokémon card. Each line corresponds to a specific card, while points on the line represent different condition grades.

Enter Phase:

- When new cards enter the filtered dataset, new group elements are created, each containing:
 1. A colored line representing the trend across conditions.
 2. Several circles marking individual condition averages.
 Lines start with standard stroke width and opacity, and circles fade in with a brief animation to enhance perceptual stability.

Update Phase:

- If the filter changes or zoom level shifts, the y-axis and all line/point positions transition smoothly. This continuous motion helps users maintain the relationship between different conditions and the global price scale. Interactive updates—such as selection or zoom slider adjustment—trigger local transitions instead of full redraws, minimizing cognitive disruption.

Exit Phase:

- When a card leaves the filtered dataset (e.g., due to filter changes), its line and associated points gradually fade out before removal. This ensures that disappearing elements are perceived as intentional removals rather than flickering artifacts.

Interaction Features:

- Hovering over a line displays a tooltip summarizing the card's metadata and price sequence.
- Hovering over points reveals condition-specific prices.
- Clicking a line or point selects that card; Ctrl/Cmd click adds or removes cards from the comparison set.
- A **zoom slider** adjusts the y-axis domain, allowing detailed inspection of small price differences without losing global context.
- Mouse-wheel panning moves the visible window vertically, offering additional control over scale focus.

Design Rationale:

Smooth transitions and adaptive scaling maintain the viewer's perceptual anchor, critical for comparing slopes and relative distances between lines.

Using opacity and stroke width changes for highlights ensures non-intrusive yet clear feedback.

Chart Integration

The integration model from CP III (shared data, linked views, global deselect) is maintained, but extended to cover the new idioms.

New in CP IV:

- **Bidirectional Linking:** Selecting a card in the bar chart or line chart now updates the scatterplot and vice versa.
- **Multi-selection Support:** Users can now select multiple cards simultaneously, with all selected items highlighted across views.

The mechanism remains modular: each chart listens to the same global `selectedCards` array, ensuring that future idioms (e.g., violin plot) will integrate without structural changes.